

计算理论与算法分析设计

教材:

[1][王] 王晓东,计算机算法设计与分析(**第5/6版**),电子工业出版社.

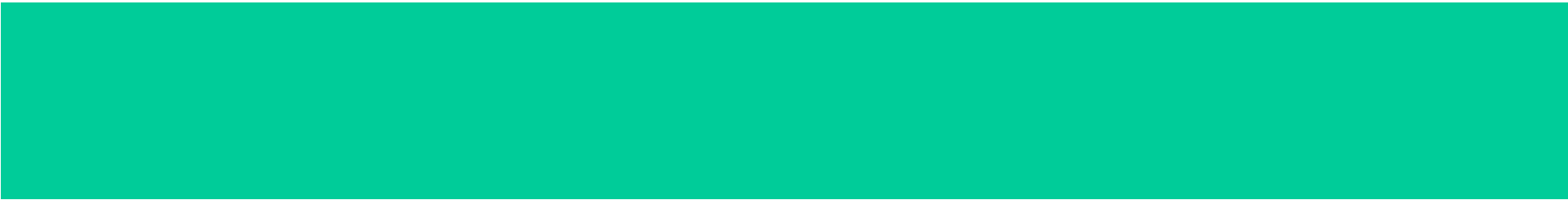
[2][S] 段磊、唐常杰等译, **Sipser**著, 计算理论导引, 机械工业出版社.

参考资料:

[3][C] 潘金贵等译, **Cormen**等著, 算法导论, 机械工业出版社.

[4][M] 黄林鹏等译, **Manber**著, 算法引论-一种创造性方法, 电子工业出版社.

[5][L] **Lewis**等著, 计算理论基础, 清华大学出版社.



第1章 算法概述

引言

旅行商问题 **Traveling Salesman Problem (TSP)**:

设有 n 个城市，已知任意两城市之间距离，现有一推销员想从某一城市出发巡回经过每一城市（且每城市只经过一次），最后又回到出发点，问如何找一条最短路径。

穷举法的时间复杂性为 $O(n!)$

$n = 21$ 时， $21! = 5.11 \times 10^{19}$ ，设每条路径需 CPU 时间为 $1ns$ ，则总共要 1620 多年才能完成。

引言

- 算法的五个特征

- 1) 有穷性
- 2) 确定性
- 3) 输入
- 4) 输出
- 5) 可行性

- 算法的定义:

- Informally, an *algorithm* is any well-defined computational procedure that takes some value, or set of values, as *input* and produces some value, or set of values, as *output*. An algorithm is thus a sequence of computational steps that transform the input into the output.

引言： 1.1 算法的定义和特征

- 算法

— 算法是在有限步骤内求解某一问题所使用的一组定义明确的规则。



解决某问题所用到的各种运算的序列。

引言： 1.1 算法的定义和特征

- 算法的特征

(1) 输入

一个算法有0个或多个输入，它是由外部提供的，作为算法开始执行前的初始值，或初始状态。算法的输入是从特定的对象集合中抽取的。

(2) 输出

一个算法有一个或多个输出，这些输出和输入有特定的关系，实际上是输入的某种函数。不同取值的输入，产生不同结果的输出。

引言： 1.1算法的定义和特征

- 算法的特征

- (3) 有限性

- 算法在执行有限步之后必须终止。

- (4) 确定性

- 算法的每一个步骤，都有精确的定义。要执行的每一个动作都是清晰的、无歧义的。

算法中不允许出现诸如“计算 $5+3$ ”
或计算“ $7-3$ ”这样的指令

引言： 1.1算法的定义和特征

- 算法的特征

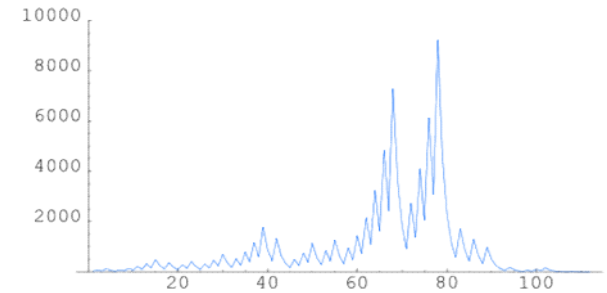
- (5) 能行性

- 算法的能行性指的是算法中有待实现的运算，都是基本的运算。原则上可以由人们用纸和笔，在有限的时间里精确地完成。
 - 要求一条算法指令应当最终能够由执行有限条计算机指令来实现。

例如，一般的整数算法运算是能行的，但如果 $1 \div 3$ 的值需由无穷的十进制展开的实数表示，就不是能行的。

$3n+1$ 问题

- 输入: 一个正整数 n ,
- 映射: $f(n) = n/2$, 若 n 是偶数;
 $f(n) = 3n+1$, 若 n 是奇数.
- 迭代: $5 \rightarrow 16 \rightarrow 8 \rightarrow \dots$, 到1则停止
- 输出: n 可在 f 迭代下是否能到1停止
- 直接模拟是正确的算法吗?
- 27需迭代111步(见右图)
- $1 \sim 5 \times 10^{18}$ 都能到1 ([wiki]), 即从1到 5×10^{18} 之间的每一个整数, 按照 $3n+1$ 规则迭代, 最终全都能回到 1



3n+1问题

- 70年代中期，美国各所名牌大学校园内，人们都像发疯一般，夜以继日，废寝忘食地玩弄一种数学游戏。这个游戏十分简单：
- 任意写出一个自然数N，并且按照以下的规律进行变换：如果是个奇数，则下一步变成 $3N+1$ 。如果是个偶数，则下一步变成 $N/2$ 。
- 因为人们发现，无论N是怎样一个数字，最终都无法逃脱回到谷底1。准确地说，是无法逃出落入底部的4-2-1循环，永远也逃不出这样的宿金。
- 这就是著名的“冰雹猜想”，又被称为角谷猜想。

2022年1月26日，美国世界开放性数学刊物《Advances In Pure Mathematics》(《纯粹数学进展》)在2022年第12卷第1期发表了大学教师王茂泽与三位合作者的文章《The Proof of The $3x+1$ Conjecture》(《 $3x+1$ 猜想的证明》)。至此，世界公认的著名数论难题得以破解！用化归的思想方法彻底证明该猜想是正确的。

不可判定问题(没有算法)举例

Hilbert第十问题: “多项式是否有整数根”有没有算法?

1970's 被证明不可判定.

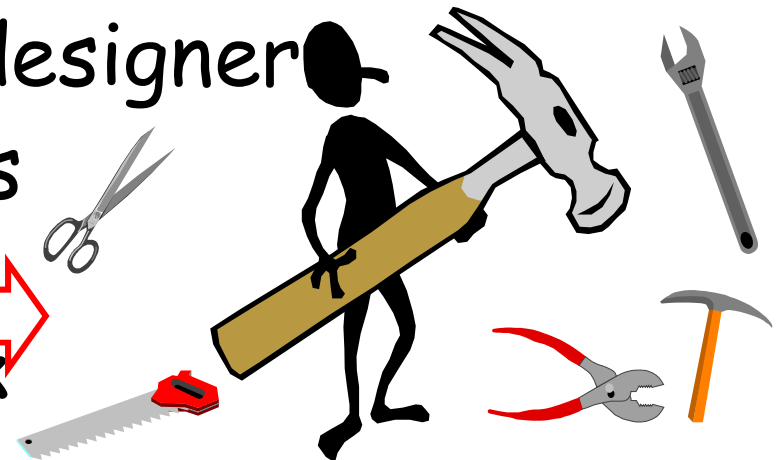
M = “对于输入 “**p**”, **p**是**k**元多项式,

1. 取**k**个整数的向量**x** (绝对值和从小到大)
2. 若 $p(x) = 0$, 则停机接受.
3. 否则转1.”

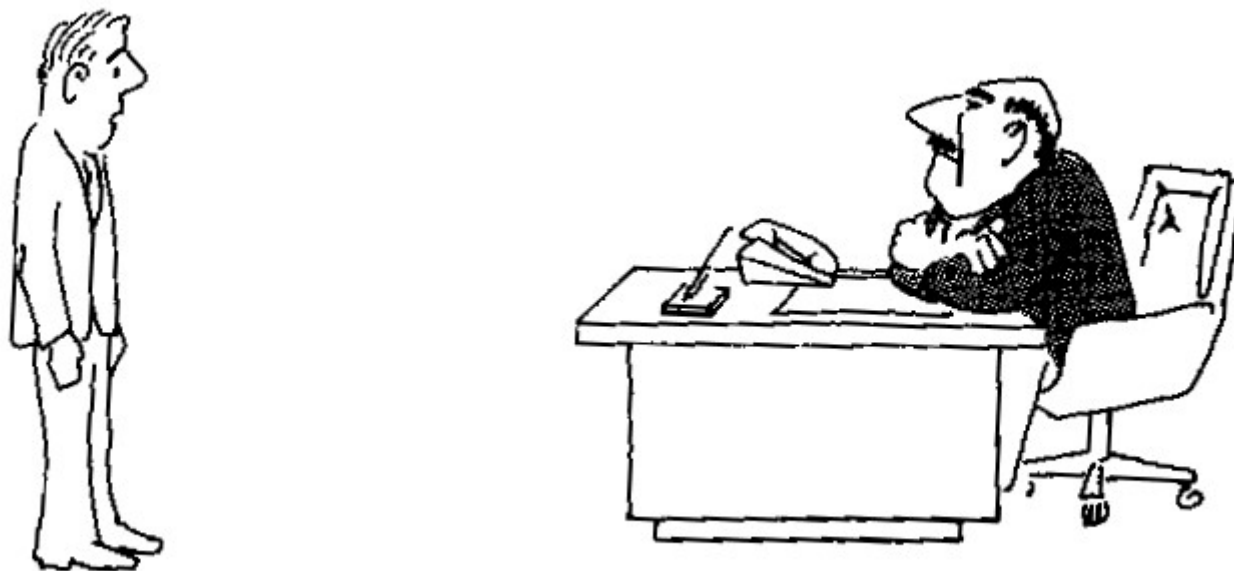
这个图灵机对输入 $p(x,y) = x^2 + y^2 - 3$ 不停机

引言

- A survey of algorithmic design techniques.
- Abstract thinking.
- How to develop new algorithms for any problem that may arise.
- Be a great thinker and designer
- Not: A list of algorithms
 - - Learn their code
 - - Trace them until work
 - - Implement them
 - - be a mundane programmer



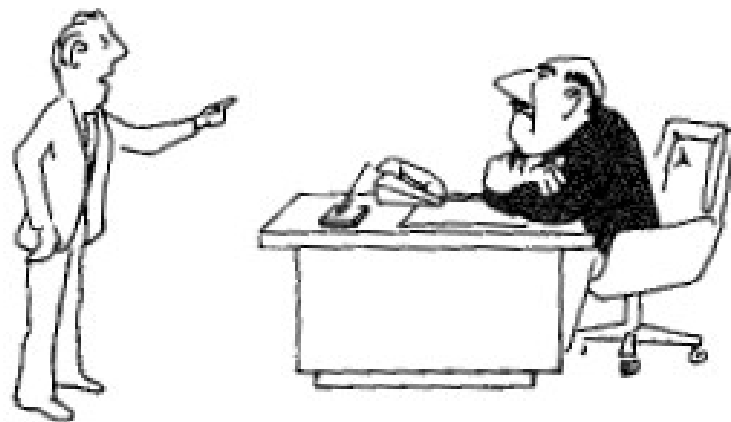
引言：学习算法的意义



"I can't find an efficient algorithm, I guess I'm just too dumb."

Serious damage to your
position within the company !!!

引言：学习算法的意义



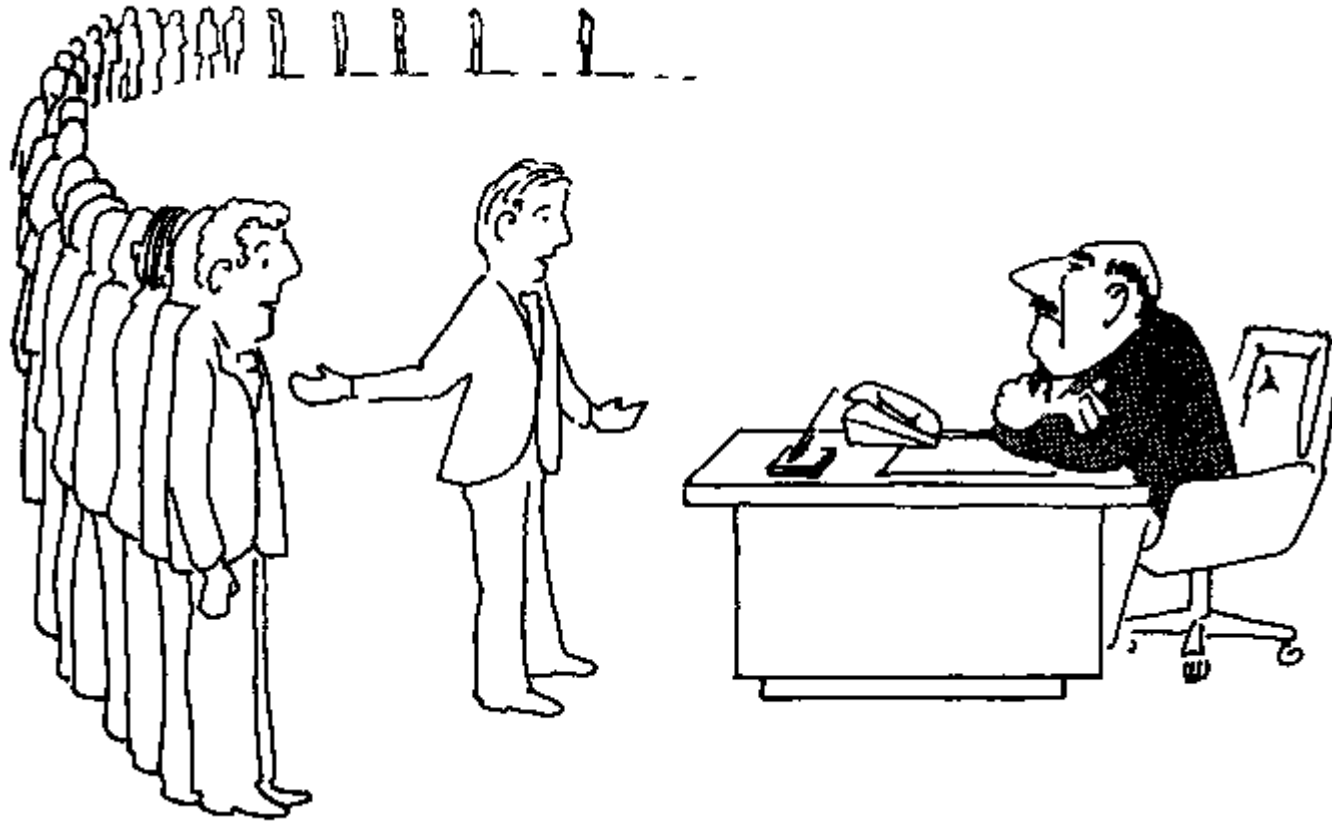
"I can't find an efficient algorithm, because no such algorithm is possible!"

Unfortunately, proving intractability can be just as hard as finding efficient algorithms !!!

$\therefore$ No hope !!!

$P \neq NP$

引言：学习算法的意义



"I can't find an efficient algorithm, but neither can all these famous people"

引言：学习算法的意义



"Anyway, we have to solve this problem.
Can we satisfy with a good solution ? "

一些有趣的问题

- (1) 背包问题1：有一人要从 n 种物品中选取不超过 b 千克重的行李随身携带，要求总价值最大。
例：设背包的容量为50千克。物品1重10千克，价值60元；物品2重20千克，价值100元；物品3重30千克，价值120元。求总价值最大。
- (2) 背包问题2：有一人要从 n 种货物中选取不超过 b 千克重的行李随身携带，要求总价值最大。
例：设背包的容量为50千克。物品1有60千克，每千克价值60元；物品2有20千克，每千克价值100元；物品3有40千克，每千克价值120元。求总价值最大。

The Drunk Jailer

(3) A certain prison contains a long hall of n cells, each right next to each other. Each cell has a prisoner in it, and each cell is locked. One night, the jailer gets bored and decides to play a game. For round 1 of the game, he takes a drink of whiskey, and then runs down the hall unlocking each cell. For round 2, he takes a drink of whiskey, and then runs down the hall locking every other cell (cells 2, 4, 6, ...). For round 3, he takes a drink of whiskey, and then runs down the hall. He visits every third cell (cells 3, 6, 9, ...). If the cell is locked, he unlocks it; if it is unlocked, he locks it.

The Drunk Jailer

He repeats this for n rounds, takes a final drink, and **passes out.**

Input: The first line of input contains a single positive integer. This is the number of lines that follow. Each of the following lines contains a single integer between 5 and 100, inclusive, which is the number of cells n .

Output: For each line, print out the number.

样例输入 样例输出

2 2

5 10

100

喝醉的狱卒

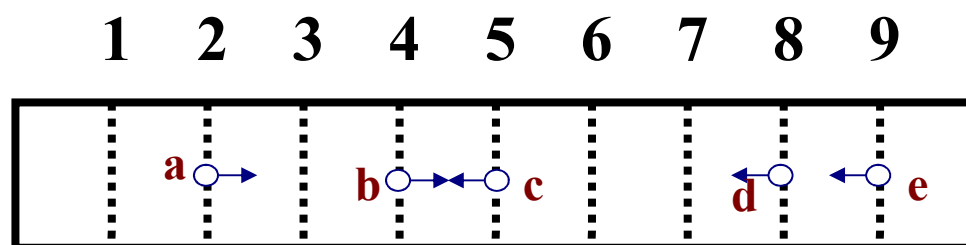
- 某个监狱的大厅里有 n 个牢房，每个牢房彼此相邻。每个牢房都有一个囚犯，每个牢房都被锁上了。
- 一天晚上，狱卒感到无聊并决定玩游戏。第一轮比赛，他喝了一杯威士忌，然后跑到大厅里打开每个牢房。第2轮，他喝了一杯威士忌，然后跑到大厅里把所有牢房（牢房2,4,6, ...）锁上。对于第3轮，他喝了一杯威士忌，然后跑到大厅，他每三个牢房操作一次（牢房3,6,9,）。如果牢房被锁住，他会将其打开；如果它被打开了，他会把它锁上。

喝醉的狱卒

- 他重复了 n 轮，最后喝了一杯，然后昏倒了。一些囚犯，可能是零，意识到他们的牢房被打开，狱卒无法行动。他们立即逃脱。给你牢房的数量，确定有多少囚犯逃离监狱。
- 输入：第一行输入包含一个正整数。表示后面输入的行数。以下每行包含一个5到100之间的整数，表示有 n 个牢房。
- 输出：对于每一行，您必须打印出监狱有 n 个牢房时逃脱的囚犯数量。
- 样例输入 样例输出
- 2 2
- 5 10
- 100

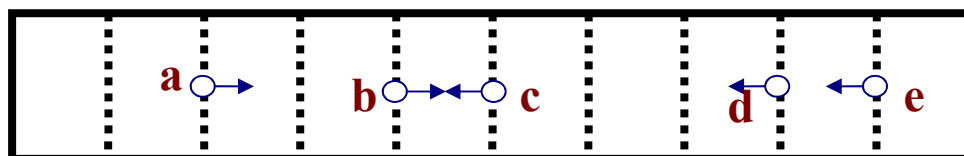
一些有趣的问题：蚂蚁问题

有一根10厘米的细木杆，在第2厘米、第4厘米、第5厘米、第8厘米、第9厘米这五个位置上各有一只蚂蚁。木杆很细，不能同时通过一只蚂蚁。开始时，蚂蚁的头朝左还是朝右是任意的，它们只能朝前走或调头，但不会后退。所有蚂蚁的速度都相同，均为 1cm/s 。当任意两只蚂蚁碰头时，两只蚂蚁会同时掉头并且仍然按 1cm/s 朝相反方向走。求蚂蚁掉下木杆的最小和最大时间。

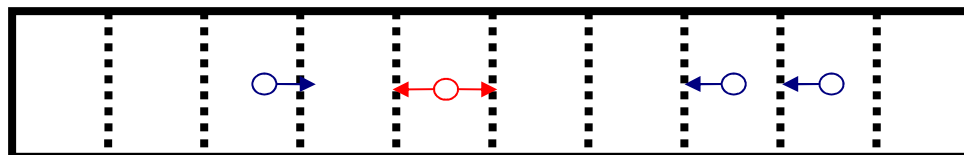


一些有趣的问题： 蚂蚁问题

1 2 3 4 5 6 7 8 9

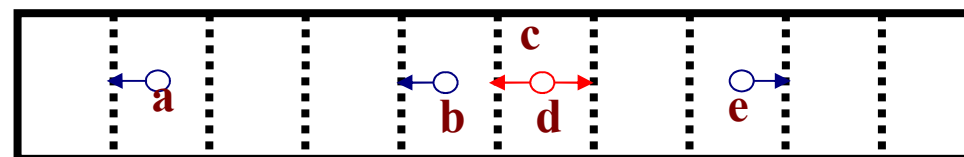


0.5s



...

3.5s



5秒， a掉； 6秒， e掉； 8秒， b和d掉； 9秒， c掉。

1.2 算法的评价

评价算法的标准：

1) 正确性(Correctness)

-- 在合法的输入下，算法能够实现预先规定的功能和计算精度要求。

2) 效率(Efficiency)

-- 算法执行的时间短（时间复杂性）

-- 算法需要的存储空间小（空间复杂性）

1.2 算法的评价

评价算法的标准：

3) 可读性/简明性

--读者对于源代码的功能意图、流程控制和操作运行是否容易把握。

---要求算法的逻辑清晰、简单明了、并且是结构化的，从而使算法易于阅读和理解，并易于编码和测试。

1.2 算法的评价

- 可读性重要的原因
 - 程序员会把大部分时间，花费在阅读并试图理解和修改现存源代码上面，而不是编写新的源代码。
 - 没法读的代码往往导致缺陷、低效与代码重复。
 - 一点点简单的可读性改造，也能让代码变得简短，并且大大缩短看懂所需的时间(就像一段没有善加利用标点符号，部分带有冗赘词语的文句。稍微修改该文句，改以适当的方式使用标点符号，将冗赘词语修正。就能提高读者读取文句讯息的流畅度)。

1.2 算法的评价

- 遵循固定的编程风格往往会改善可读性
 - 不同的缩进风格（空白）
 - 注释
 - 任务分解
 - 命名约定：用于各种对象，诸如变量、类、子程序等

1.2 算法的评价

评价算法的标准:

4) 最优性

— 算法的执行时间已达到求解该类问题所需时间的下界。

1.2 算法的评价

- 影响程序运行时间的因素
 - (1) 计算机系统性能;
 - (2) 程序所依赖的算法;
 - (3) 问题规模和输入数据;
 - 问题的规模一般是指输入数据的量, 必要时考虑输出数据的量。
 - 问题的规模相同, 输入数据的状态不同, 所需的时间开销也会不同。

课程相关

- 课程内容
 - ✓ 递归与分治、动态规划、贪心法、回溯法
 - ✓ 正则语言、图灵论题、可判定性、可归纳性
- 课程安排
 - 32学时授课
- 成绩
 - 平时成绩**30%**， 期末闭卷成绩**70%**

时间复杂性

- (1) 假设某算法在输入规模为 n 的计算时间为 $T(n)=3*2^n$ 。在某台计算机上实现并完成该算法的时间为 t 秒。现在另一计算机，其运行速度为第一台的64倍，那么在这台新机器上用同一算法在 t 秒内能解输入规模为 $n+5$ 的问题？

时间复杂性

第一台计算机: $t = 3 * 2^n$

第二台计算机: $t / 64 = 3 * 2^n,$

$$\text{即 } 3 * 2^{n_1} / 64 = 3 * 2^n$$

解: 设新机器用同一算法内能解输入规模为 n_1 的问题, 那么有 $t=3 * 2^n=3 * 2^{n_1} / 64$, 解得 $n_1=n+6$.

时间复杂性

- 例：算法 A_1, A_2 的时间复杂性分别是 $n, 2^n$ ，设 $100\mu s$ 是一个单位时间，求 A_1, A_2 在 $1s$ 内能处理的问题规模。
- 已知 $\lg 2 = 0.301$

$$T(n) = n$$

$$T(n) * 10^{-4} = 1 \quad \text{对于 } 2^n, \quad n=13.$$

$$\text{即 } n * 10^{-4} = 1$$

$$\text{所以 } n = 10^4$$

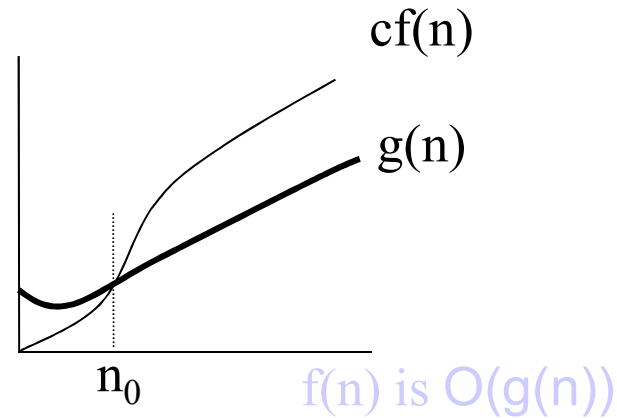
算法运行时间的评估

- 计算机速度提高**10**倍后，不同算法复杂性求解规模的扩大情况

算法	A1	A2	A3	A4	A5	A6
时间复杂性	n	$n \log n$	n^2	n^3	2^n	$n!$
n_2 和 n_1 的关系	$10 n_1$	$8.38 n_1$	$3.16 n_1$	$2.15 n_1$	$n_1 + 3.3$	n_1

n_1 为计算机速度提高前处理的问题规模
 n_2 为计算机速度提高后处理的问题规模

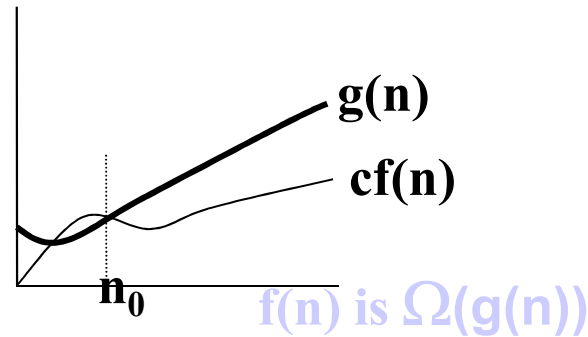
运行时间的上界：O记号



- O记号的定义：**

令 $\mathbf{N}$ 为自然数集合， $\mathbf{R}_+$ 为正实数集合。函数： $\mathbf{f}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，函数： $\mathbf{g}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，若存在自然数 $\mathbf{n}_0$ 和正常数 $\mathbf{c}$ ，使得对所有的 $\mathbf{n}\geq\mathbf{n}_0$ ，都有 $\mathbf{g(n)}\leq\mathbf{cf(n)}$ ，就称函数的阶至多是 $\mathbf{O(f(n))}$ 。

运行时间的下界： Ω 记号

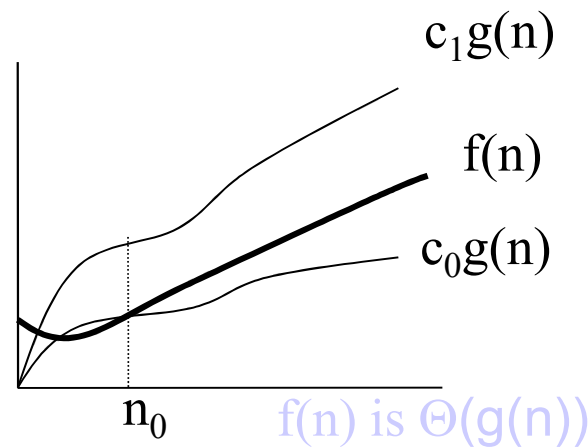


- Ω 记号的定义

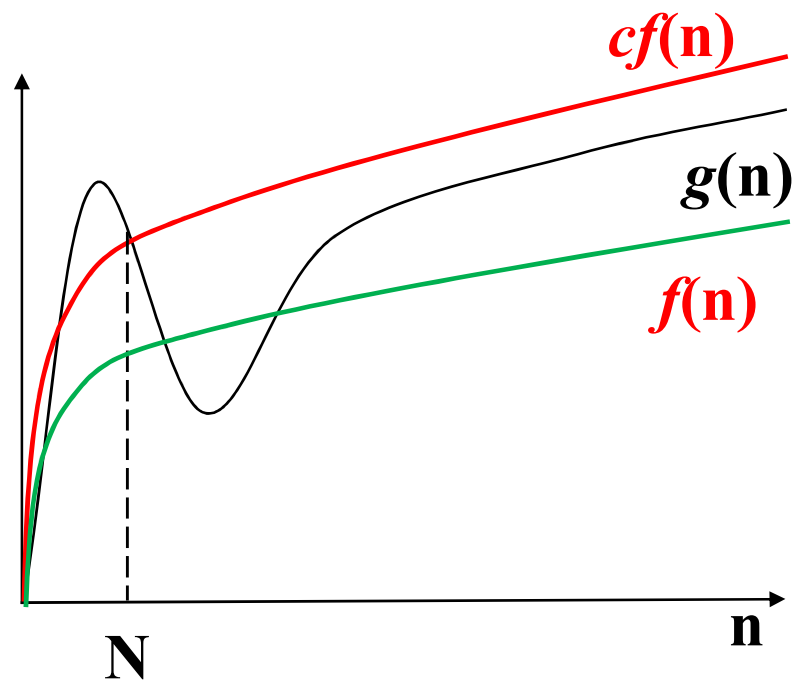
令 $\mathbf{N}$ 为自然数集合， $\mathbf{R}_+$ 为正实数集合。函数： $\mathbf{f}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，
函数： $\mathbf{g}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，若存在自然数 $\mathbf{n}_0$ 和正常数 $\mathbf{c}$ ，使得
对所有的 $\mathbf{n}\geq\mathbf{n}_0$ ，都有 $\mathbf{g}(\mathbf{n})\geq\mathbf{c}\mathbf{f}(\mathbf{n})$ ，就称函数的阶至少是 $\Omega(\mathbf{f}(\mathbf{n}))$ 。

运行时间的准确界： Θ 记号

◆定义：令 $\mathbf{N}$ 为自然数集合， $\mathbf{R}_+$ 为正实数集合。
函数： $\mathbf{f}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，函数： $\mathbf{g}:\mathbf{N}\rightarrow\mathbf{R}_+$ ，若存在自然数 $\mathbf{n}_0$ 和正常数 $\mathbf{c}_1, \mathbf{c}_2$ ，使得对所有的 $\mathbf{n}\geq\mathbf{n}_0$ ，都有 $\mathbf{c}_1\mathbf{f}(\mathbf{n})\leq\mathbf{g}(\mathbf{n})\leq\mathbf{c}_2\mathbf{f}(\mathbf{n})$ ，就称函数的阶是 $\Theta(\mathbf{f}(\mathbf{n}))$ 。



算法渐进分析：大O表式法

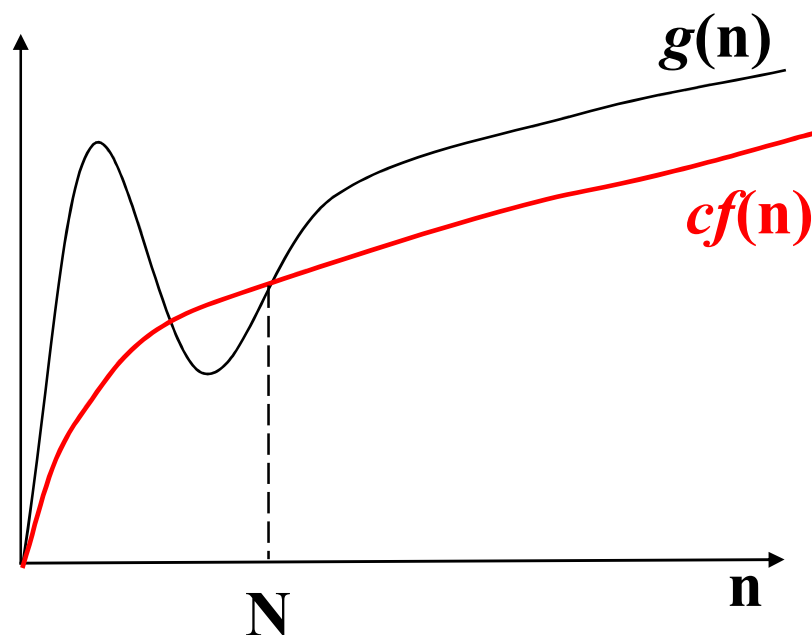


$f(n)$ 是 $g(n)$ 的紧密上限

- ◆ 该定义说明了函数 g 和 f 之间的关系：函数 $f(n)$ 是函数 $g(n)$ 取值的**上限 (upper bound)**，或说函数 g 的增长最终至多趋同于 f 的增长。
- ◆ 因此，大O表示法提供了一种表达**函数增长率上限**的方法

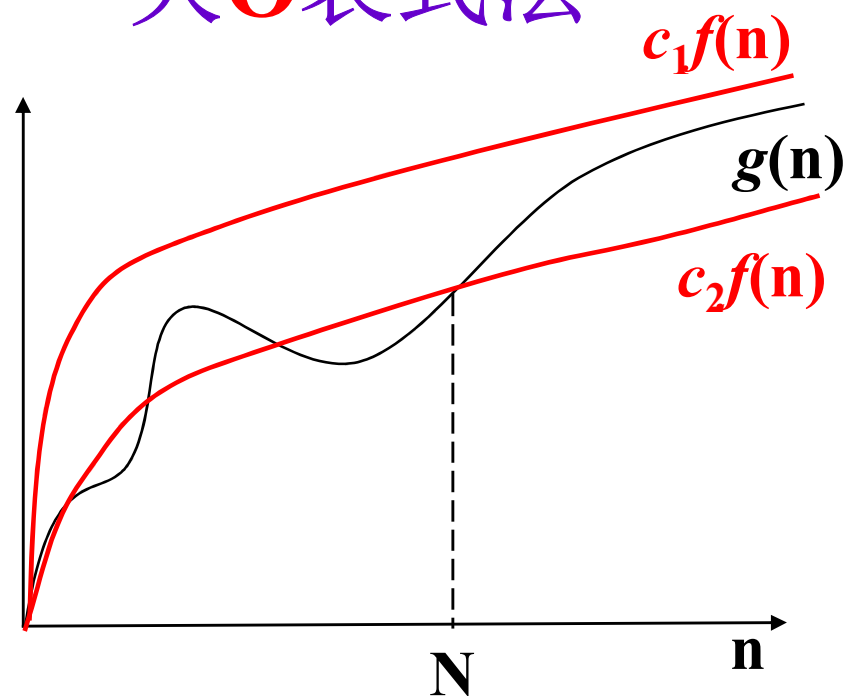
算法时间复杂度

大 Ω 表式法



$f(n)$ 是 $g(n)$ 的紧密下限

大 Θ 表式法



$f(n)$ 是 $g(n)$ 的紧确界

符号

一、符号说明

1. 取整函数

$\lfloor x \rfloor$: 小于等于 x 的最大整数

$\lceil x \rceil$: 大于等于 x 的最小整数

性质

$$x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$$

符号

$$\left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor = n$$

$$\left\lceil \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rceil = \left\lceil \frac{n}{ab} \right\rceil$$

$$\left\lfloor \frac{\left\lfloor \frac{n}{a} \right\rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor$$

符号

$$\lceil a/b \rceil \leq [a+(b-1)]/b$$

$$\lfloor a/b \rfloor \geq [a-(b-1)]/b$$

符号

- 鸽巢原理（又名抽屉原理）

若 $n+1$ 只鸽子飞入 n 个鸽巢，那么至少有一个鸽巢至少有2只鸽子。

若 n 只鸽子飞入 m ($n > m$) 个鸽巢，那么至少有一个鸽巢至少有 $\lfloor (n-1)/m \rfloor + 1$ 只鸽子。

Halloween treats

- Every year there is the same problem at Halloween: Each neighbor is only willing to give a certain total number of sweets on that day, no matter how many children call on him, so it may happen that a child will get nothing if it is too late. To avoid conflicts, the children have decided they will put all sweets together and then divide them evenly among themselves. From last year's experience of Halloween they know how many sweets they get from each neighbor. Since they care more about justice than about the number of sweets they get, they want to select a subset of the neighbors to visit, so that in sharing every child receives the same number of sweets. They will not be satisfied if they have any sweets left which cannot be divided.

Halloween treats

Sample Input

4 5 (the number of children and the number of
neighbors, respectively.)

1 2 3 7 5

3 6

7 11 2 5 13 17

0 0

Sample Output

3 5

2 3 4

符号

- 2 对数

$$\log n = \log_2 n, \quad \lg n = \log_{10} n \quad \lg 2 = 0.301$$

$$\log^k n = (\log n)^k$$

$$\log \log n = \log(\log n)$$

性质：

$$a^{\log_b n} = n^{\log_b a}$$

$$\log_k n = c \log_l n$$

符号

- 证明:

$$\begin{aligned} a^{\log_b n} &= b^{\log_b a^{\log_b n}} \\ &= b^{\log_b n \cdot \log_b a} \\ &= (b^{\log_b n})^{\log_b a} \\ &= n^{\log_b a} \end{aligned}$$

符号

- $\log_b N = \log_a N / \log_a b$
- $\log_e N = \ln N$
- $e = 2.71828$
- $\log_a b = 1 / \log_b a$

符号

- 二、阶乘

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \omega(2^n)$$

$$\log n! = \Theta(n \log n)$$

符号

- 三、求和
等比级数的求和公式

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$

分治算法

主要内容:

- 1. 递归函数
- 2. 分治思想、适用条件
- 3. 分治原理、主定理、二分法
- 4. 应用问题
 - 最大最小元
 - 排序
 - 循环赛日程表
 - 大整数乘法
 - 线性时间选择
 - **Strassen**矩阵乘法
 - 最大子段和
 - 棋盘覆盖
 - 最接近点对问题

递归函数

- **递归算法**: 直接或间接地调用自身的算法
- **递归函数**: 用函数自身给出定义的函数
- 边界条件与递归方程是递归函数的两要素
保障在有限次计算后得出结果

$$n! = \begin{cases} 1 & n = 0 \text{ 边界条件} \\ n(n-1)! & n > 0 \text{ 递归方程} \end{cases}$$

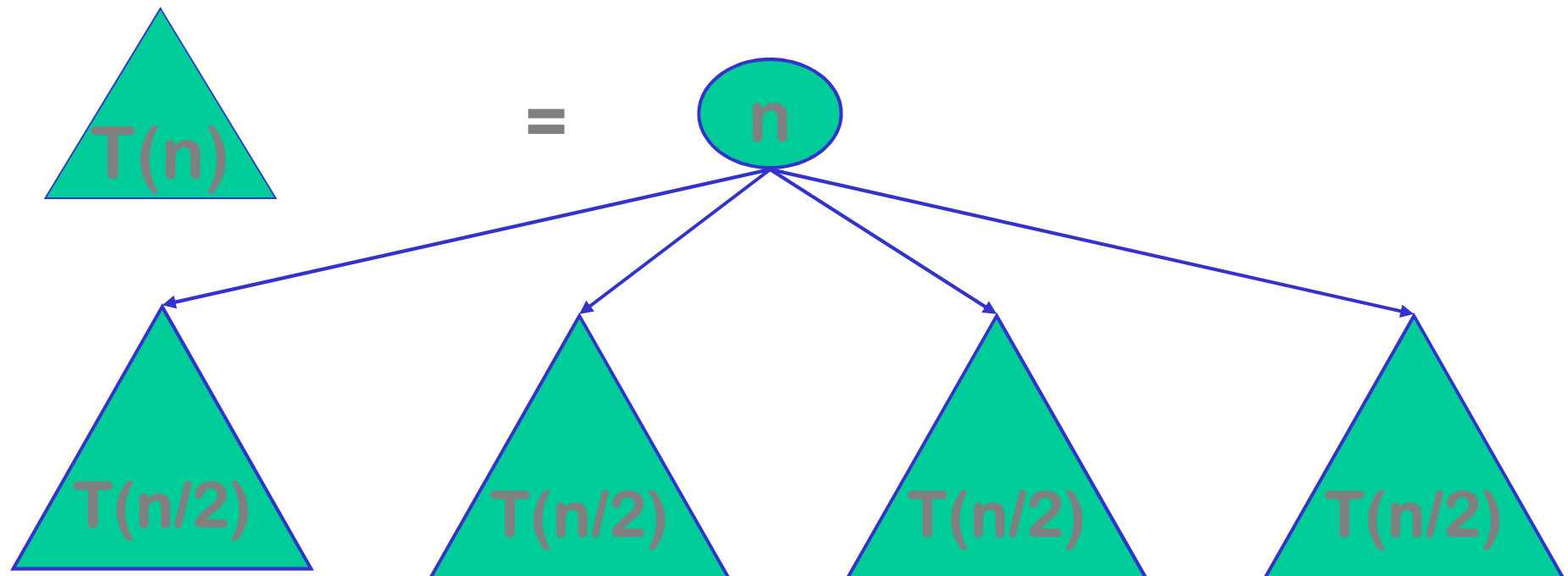
$$fib(n) = \begin{cases} 1 & n \leq 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

```
int fib(int n)
{ if (n <= 1) return 1;
  return fib(n-1)+fib(n-2); }
```

- 
- 分治思想和适用条件

分治算法总体思想(1)

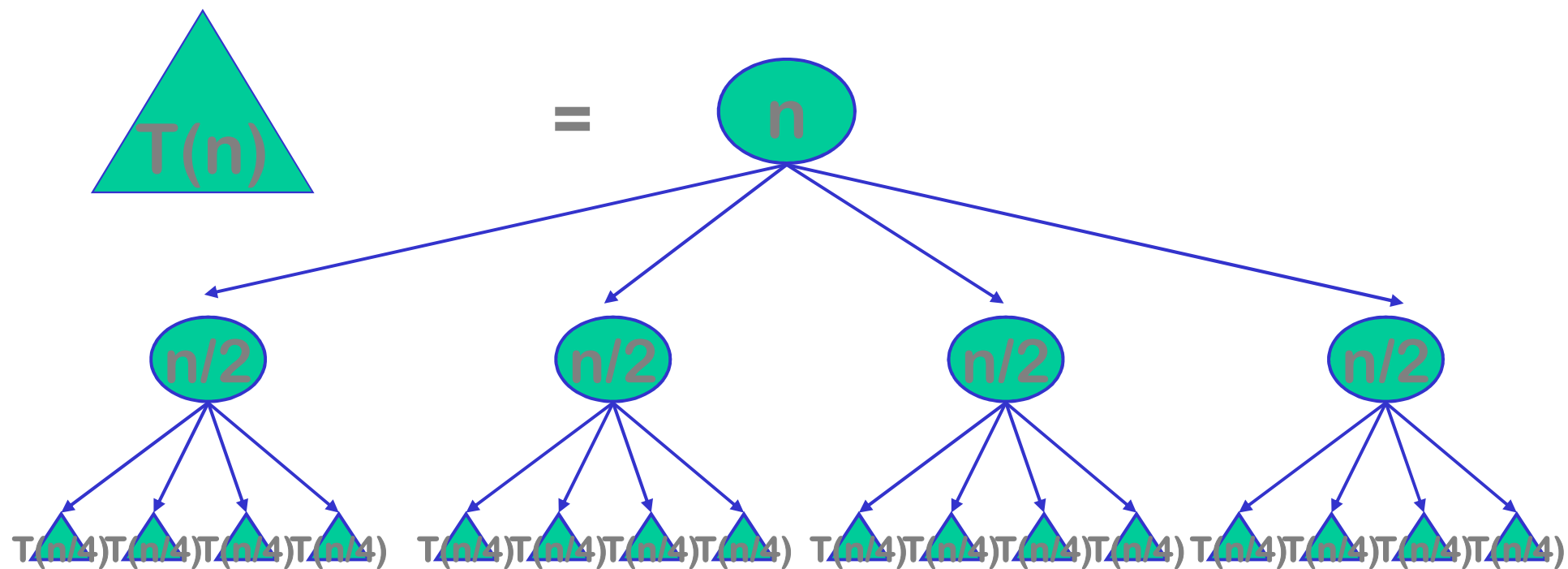
- 将要求解的较大规模的问题分割成 k 个更小规模的子问题。



分治算法总体思想(2)

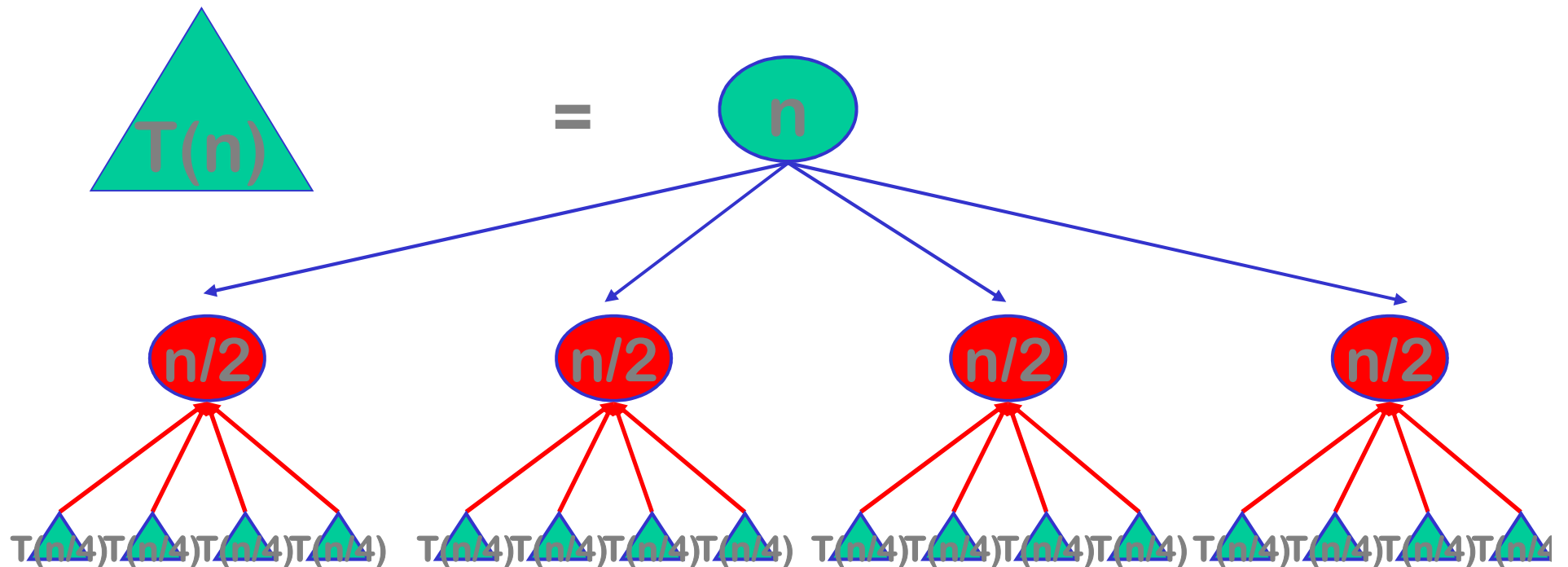
- 对这 k 个子问题分别求解。如果子问题的规模仍然不够小，则再划分为 k 个子问题，如此递归的进行下去，直到问题规模足够小，很容易求出其解为止。

分治算法总体思想

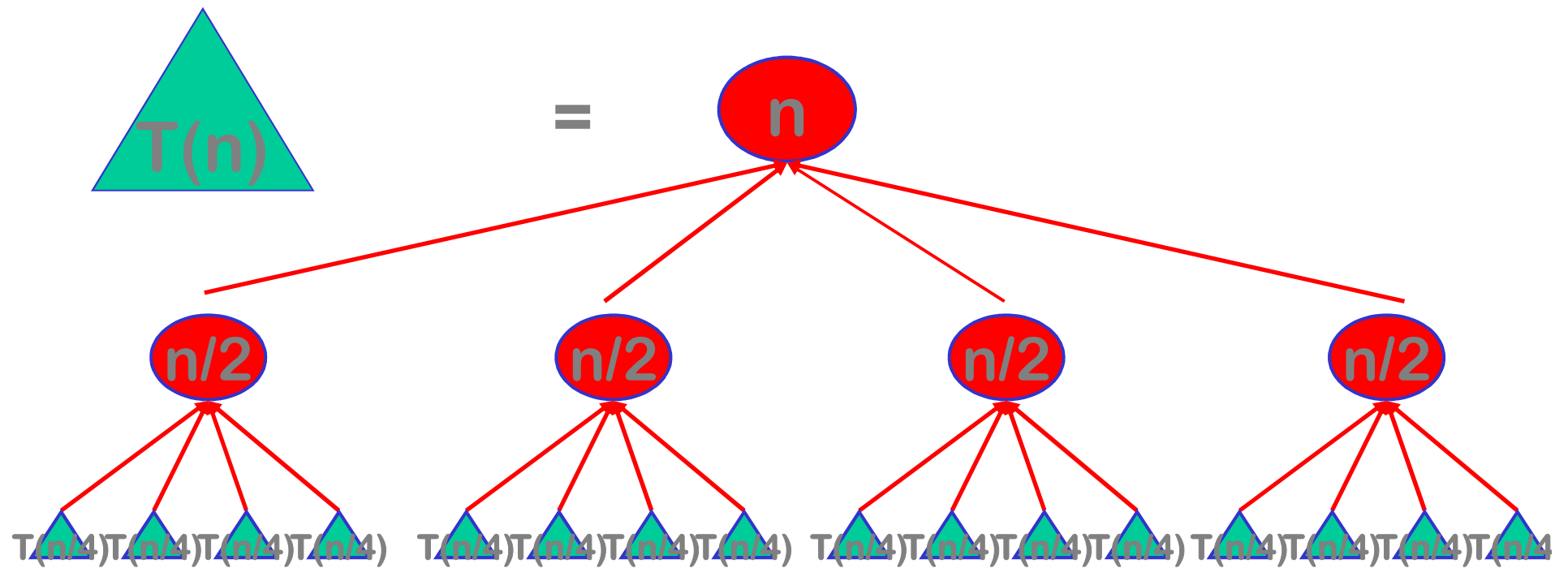


分治算法总体思想(3)

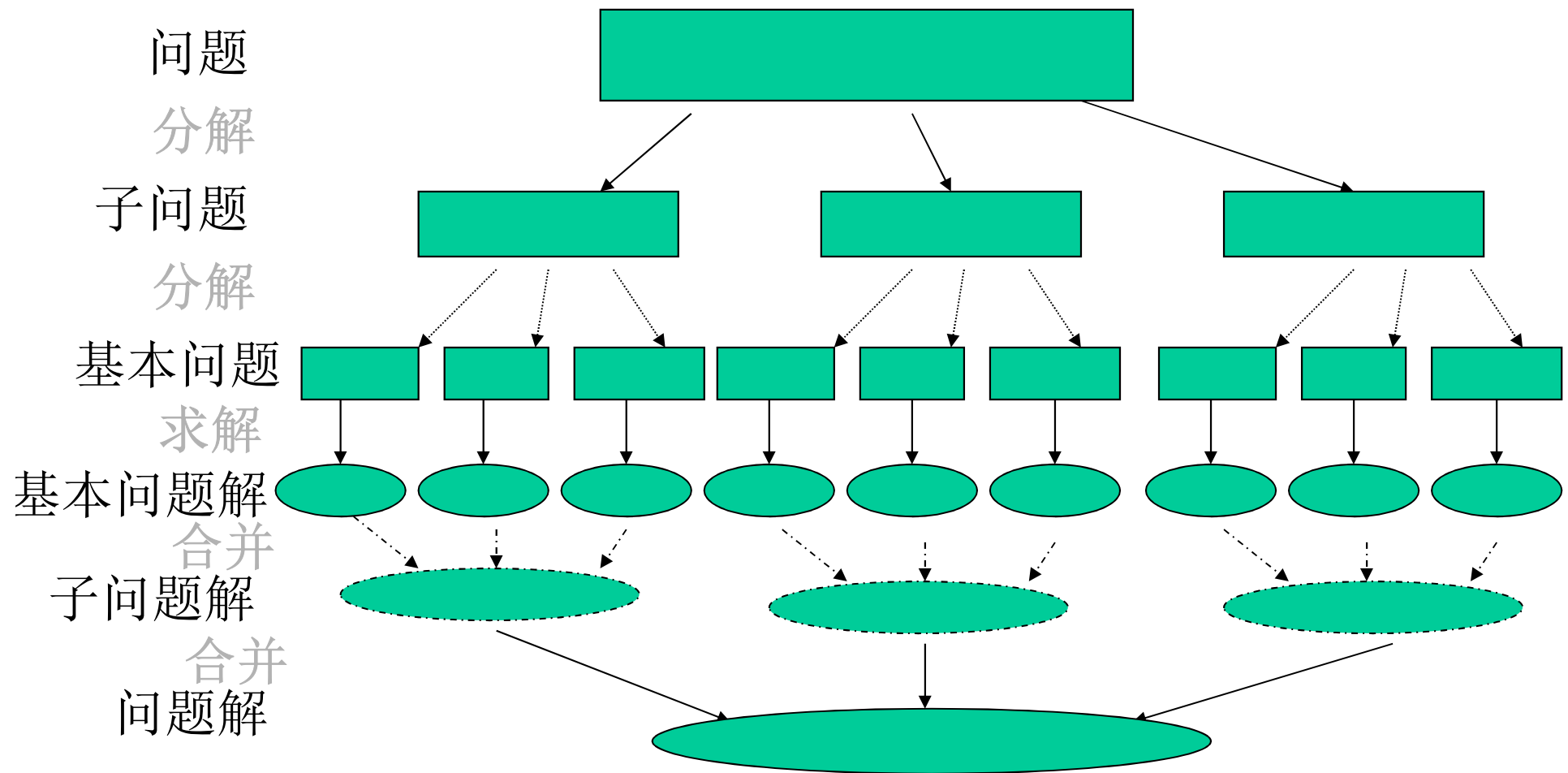
- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。



分治算法总体思想



分治算法总体思想



分治算法总体思想

分治法的设计思想是，将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。

分治算法总体思想

- 分治法是一个**递归**地求解问题的过程
- 分治法在每一层递归上有三个步骤

分解：通过某种方法，将原问题分解成若干个规模较小，**相互独立**，与原问题形式相同的子问题

解决：若子问题规模小到可以直接解决（称为基本问题），则直接解决，否则把子问题进一步分解，递归求解

合并：通过某种方法，将子问题的解合并为原问题的解

分治基本过程

分: 将问题分解成若干个子问题

治: 递归求解子问题

合: 由子问题解合并得到原问题解

divide-and-conquer(P)

```
{ if ( | P | <= n0) adhoc(P);    //解决小规模的问题
  else
  { divide P into P1, P2,..., Pa;    //分解问题
    for (i=1,i<=a,i++)
      yi=divide-and-conquer(Pi); //递归的解各子问题
    return merge(y1,...,ya);    //合并出原问题的解
  }
}
```

分治法的基本步骤

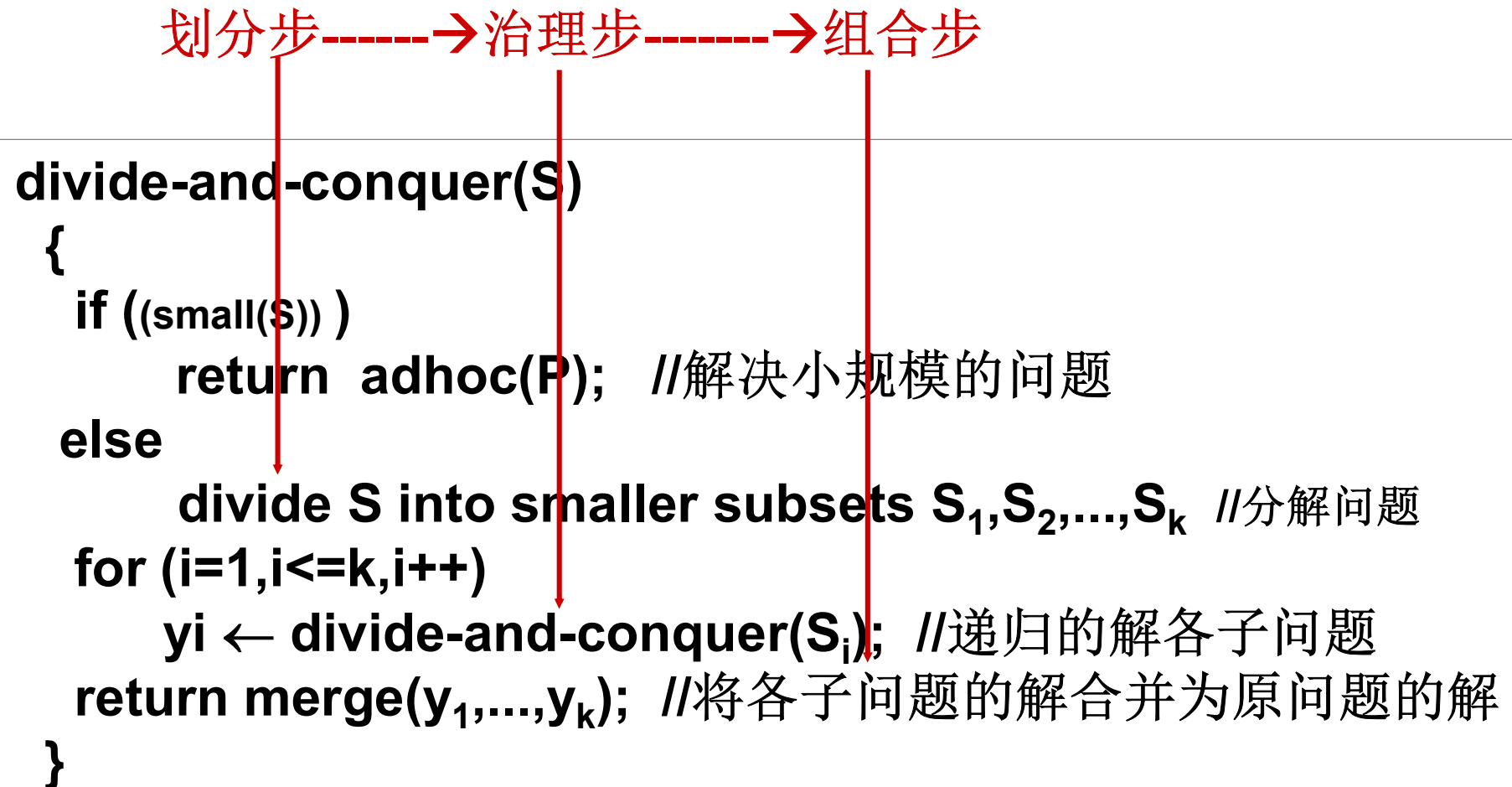
分: 将问题分解成若干个子问题

治: 递归求解子问题

合: 由子问题解合并得到原问题解

划分步-----→治理步-----→组合步

```
divide-and-conquer(S)
{
  if ((small(S)) )
    return adhoc(P); //解决小规模的问题
  else
    divide S into smaller subsets  $S_1, S_2, \dots, S_k$  //分解问题
    for (i=1, i<=k, i++)
       $y_i \leftarrow \text{divide-and-conquer}(S_i)$ ; //递归的解各子问题
    return merge( $y_1, \dots, y_k$ ); //将各子问题的解合并为原问题的解
}
```



分治法的基本步骤

- **small(S)**是一个布尔值函数，它判断**S**的输入规模是否小到无需进一步分治就能算出其答案。
- **adhoc(S)**是分治法的求解子算法。
- **merge($y_1, y_2, \dots, y_i, \dots, y_k$)**为解的归并子算法。
- 注意：应用分治法的两个前提是问题的可分性和解的可归并性，这两个前提缺一不可。

分治的原则

- 子问题相互独立(为什么), 无重复
若有大量重复子问题, 改用动态规划
- 子问题规模(n/b)大致相等(平衡思想)
- 子问题和原问题类似, 可递归求解
- 子问题解合并能得到原问题解

分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 1) 该问题的规模缩小到一定的程度就可以容易地解决；
- 2) 该问题可以分解为若干个规模较小的相同问题
- 3) 利用该问题分解出的子问题的解可以合并为该问题的解；
- 4) 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

特征分析

特征1：是绝大多数问题都可以满足的，因为问题的复杂性一般是随问题的规模增加而增加的

特征2：是分治法的前提，也是大多数问题可以满足的

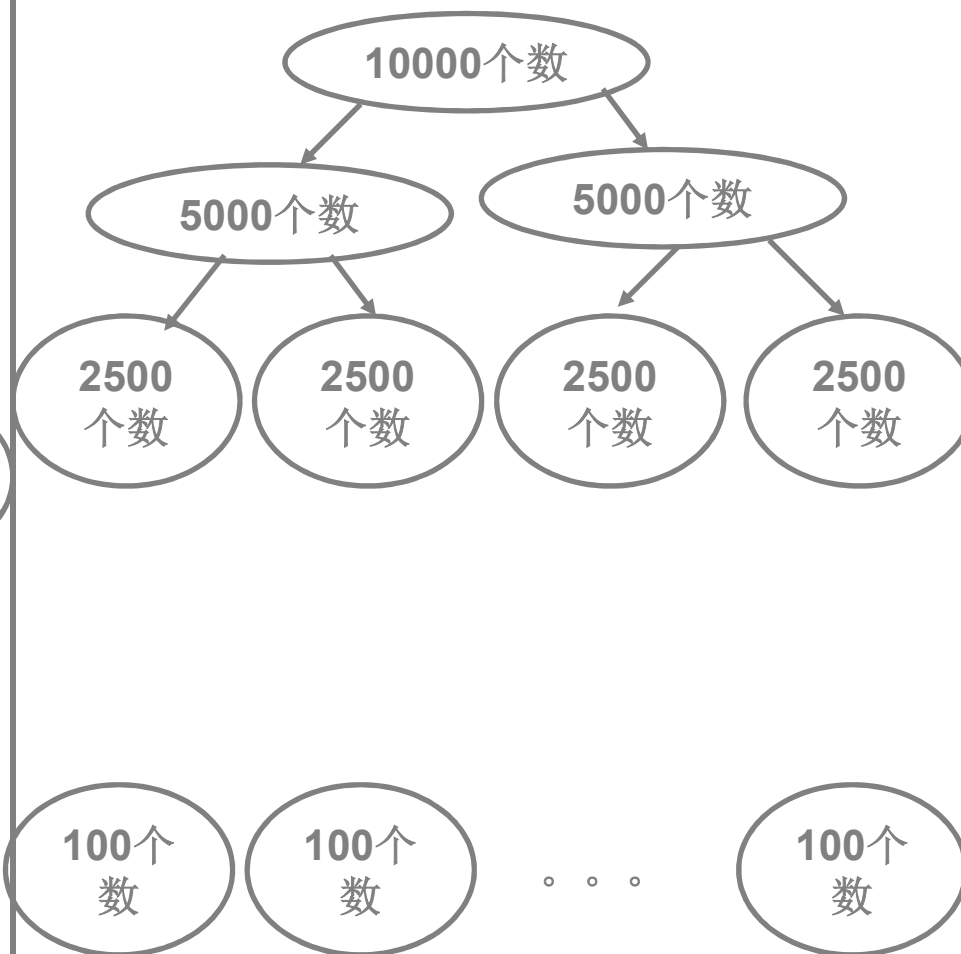
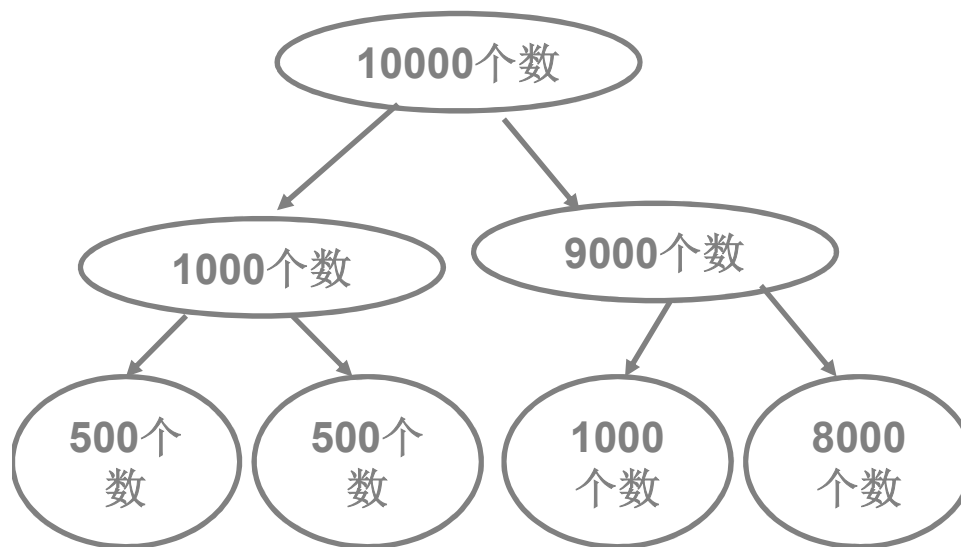
特征3：是分治法的使用的关键，能否利用分治法完全取决于能否把子问题的解合并为原问题的解

分治法的适用条件(2)

第4条特征是应用分治法的关键，此特征反映了递归思想的应用

第4条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但效率高。一般用**动态规划**较好。

例子



分治法

人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种平衡(balancing)子问题的思想，它几乎总是比子问题规模不等的做法要好。

分治法的复杂性分析

一个分治法将规模为 n 的问题分成 k 个规模为 n / m 的子问题去解。设分解阈值 $n_0=1$ ，且adhoc解规模为1的问题耗费1个单位时间。再设将原问题分解为 k 个子问题以及用merge将 k 个子问题的解合并为原问题的解需用 $f(n)$ 个单位时间。用 $T(n)$ 表示该分治法解规模为 $|P|=n$ 的问题所需的计算时间，则有：

$$T(n) = \begin{cases} O(1) & n = 1 \\ kT(n/m) + f(n) & n > 1 \end{cases}$$

$$T(n) = n^{\log_m k} + \sum_{j=0}^{\log_m n - 1} k^j f(n / m^j)$$

分治的复杂性

设分解出的子问题有 a 个, 时间复杂度 $T(n)$, 分解+合并时间 $f(n)$:

$$T(n) = \begin{cases} O(1) & n \leq n_0 \\ aT(n/b) + f(n) & n > n_0 \end{cases}$$

```
divide-and-conquer(P)
{ if ( | P | <= n0) adhoc(P);
  else
  { divide P into P1, P2,..., Pa;
    for (i=1,i<=a,i++)
      yi=divide-and-conquer(Pi);
    return merge(y1,...,ya);
  } }
```


分治中经常出现的递推关系

设 $a \geq 1$, $b \geq 2$, 分治中经常出现

$$T(n) = \begin{cases} O(1) & n \leq n_0 \\ aT(n/b) + f(n) & n > n_0 \end{cases}$$

教材中的公式 (Page 17)

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n / n_0} a^j f(n / b^j)$$

这个公式有时使用不是很方便, 介绍分治主定理

分治主定理([M]Page37)

设 $a \geq 1, b \geq 2$

$$T(n) = \begin{cases} O(1) & n \leq n_0 \\ aT(n/b) + cn^k & n > n_0 \end{cases}$$

则

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a > b^k \text{ or } k < \log_b a \\ \Theta(n^{\log_b a} \log n) & a = b^k \text{ or } k = \log_b a \\ \Theta(n^k) & a < b^k \text{ or } k > \log_b a \end{cases}$$

注:[M]中为大O记号, 无详细证明. 证明见附录.

分治主定理([C]第4章)

设 $a \geq 1, b \geq 2$

$$T(n) = \begin{cases} O(1) & n \leq n_0 \\ aT(n/b) + f(n) & n > n_0 \end{cases}$$

则

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{若 } f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0 \\ \Theta(n^{\log_b a} \log n) & \text{若 } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{若 } f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0 \end{cases}$$

注:[C]中有详细证明.

推广

$$T(n) = \begin{cases} b & n = 1 \\ T(\lfloor c_1 n \rfloor) + T(\lfloor c_2 n \rfloor) + bn & n > 1 \end{cases}$$

$$T(n) = \begin{cases} \Theta(n \log n) & c_1 + c_2 = 1 \\ \Theta(n) & c_1 + c_2 < 1 \end{cases}$$

特别地，当 $c_1 + c_2 < 1$ 时，有

$$T(n) \leq bn / (1 - c_1 - c_2) = O(n)$$

二分法

输入: 实数序列 $a_1, \dots, a_n$, 性质 P (关于序列单调)

输出: 满足性质 P 的临界点位置

例1: 输入序列($a_1 < \dots < a_n$)和 m , 判断 m 是否在序列中

枚举: 时间复杂度为 $O(n)$

二分法: 运算1次, 解范围缩小一半

$$T(n) = T(n/2) + 1$$

$$T(n) = \Theta(\log n)$$

条件: 性质 P 满足单调性

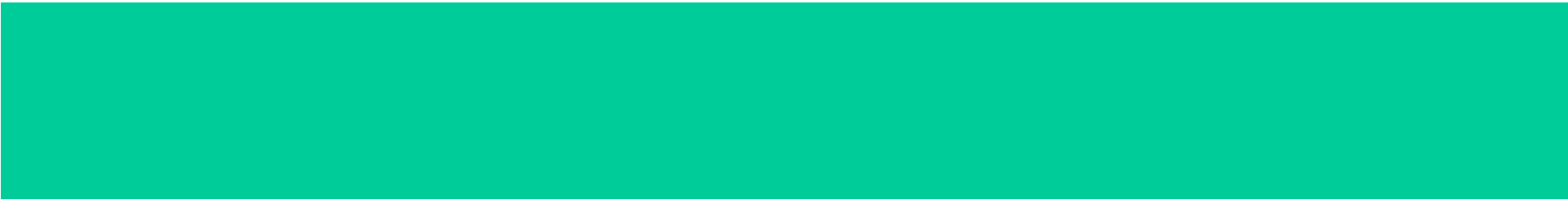
例2: 求 $f(x) = \ln x + 2x - 6$ 在 $(2, 3)$ 中的近似零点.

二分法

例3: 现给出4根电缆，长度分别为8.02、7.43、4.57、5.39，要你把它们分割成11根等长的电缆，每根电缆的最大长度是多少？

使用二分法的条件

- 解具有递增（或递减）的特性
- 对于某个值不是问题的解，那么比这个值大（或小）的值均不是问题的解



应用1：求最大最小元

问题

- 用分治法求 n 个元素集合 S 中的最大、最小元素
- 假设 $n=2^m$ 。要求每次平分成2个子集。

分治法求n元集最大最小元素

- 假设 $n=2^m$ 。要求每次平分成2个子集。

```
void maxmin(int A[],int &e_max,int &e_min,int low,int high)
2. {
3.     int mid,x1,y1,x2,y2;
4.     if ((high-low <= 1)) {
5.         if (A[high]>A[low]) {
6.             e_max = A[high];
7.             e_min = A[low];
8.         }
9.     else {
10.         e_max = A[low];
11.         e_min = A[high];
12.     }
13. }
```

分治法求n元集最大最小元素

```
14.     else {  
15.         mid = (low + high) / 2;  
16.         maxmin(A,x1,y1,low,mid);  
17.         maxmin(A,x2,y2,mid+1,high);  
18.         e_max = max(x1,x2);  
19.         e_min = min(y1,y2);  
20.     }  
21. }
```

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2)+2 & n>2 \end{cases}$$

迭代法

$$T(n) = 2T(n/2) + 2$$

$$= 2[2T(n/2^2) + 2] + 2$$

$$= 2^2 T(n/2^2) + 2(1 + 2)$$

$$= 2^3 T(n/2^3) + 2(1 + 2 + 2^2) = \dots$$

$$\text{当 } n=2^m \text{ 时, } = 2^{m-1} T(2) + 2(1 + 2 + \dots + 2^{m-2})$$

$$= 2^{m-1} + 2[1(1 - 2^{m-1}) / (1 - 2)]$$

$$= 2^{m-1} + 2^m - 2$$

$$= 3n/2 - 2$$

课堂练习

$$T(n)=3T(n/2) \quad T(1)=1 \quad n=2^m$$

$$T(n)=3^{\log_2 n}$$

答案

$$T(n)=3T(n/2) \quad T(1)=1$$

$$\begin{aligned} \textcolor{red}{n=2^k} \quad &= 3T(2^{k-1}) \\ &= 3^2T(2^{k-2}) \\ &= \dots \\ &= 3^kT(2^{k-k}) \\ &= 3^kT(1) \\ &= 3^k \end{aligned}$$

$$\textcolor{red}{k=\log_2 n}$$

课堂练习

用分治法求**n**个元素集合**S**中的最大、最小元素。
写出算法，并分析时间复杂性（比较次数）。
假设 **$n=3^m$** 。要求每次平分成**3**个子集。

$$T(n) = \begin{cases} 3 & n=3 \\ 3T(n/3)+4 & n>3 \end{cases}$$

$$T(n) = 5n/3 - 2$$

平分成**2**个子集 $T(n) = 3n/2 - 2$

迭代法

$$T(n) = 3T(n/3) + 4$$

$$= 3[3T(n/3^2) + 4] + 4$$

$$= 3^2T(n/3^2) + 4(1+3)$$

$$= 3^3T(n/3^3) + 4(1+3+3^2) = \dots$$

$$\textcolor{red}{n=3^m} \quad = 3^{m-1}T(3) + 4(1+3+\dots+3^{m-2})$$

$$= 3^m + 4[1(1 - 3^{m-1})/(1 - 3)]$$

$$= 3^m + 2 * 3^{m-1} - 2$$

$$= 5n/3 - 2$$

求n元集最大最小元素

思考题

不用（递归的）分治法求n个元素集合S中的最大、最小元素，使得

$T(n)$ 仍为 $3n/2-2$ 。

这里假设 **$n=2^m$** 。

■ 求最大最小元

其实，此问题也不一定要用递归算法来解决，可将数据分成两个一组的 $n/2$ 组，第一组比较一数，令大的为 $x_{\max}$ ，小的为 $x_{\min}$ ，以后各组比较三次，先两个数据比较，其中大者再与 $x_{\max}$ 比，小者再与 $x_{\min}$ 比，总比较次数也恰为

$$\left(\frac{3}{2}n - 2\right)$$

求最大最小元

- 注释：每组的比较次数为3次，数据共分成 $n/2$ 组，所以比较次数为 $3*n/2$ ，但是第1组的比较次数只为1次，所以比较次数为 $3*n/2-2$.

应用2：快速排序

排序

什么是排序？ Sorting

排序是计算机内经常进行的一种操作，其目的是将一组“无序”的记录序列调整为“有序”的记录序列。

例如：

52, 49, 80, 36, 14, 58, 61, 23, 97, 75

14, 23, 36, 49, 52, 58, 61, 75, 80, 97

排序

排序：

假设含 n 个记录的序列为 $\{ R_1, R_2, \dots, R_n \}$,
其相应的关键字序列为 $\{ K_1, K_2, \dots, K_n \}$,
这些关键字相互之间可以进行比较, 即在它们之
间存在着这样一个关系: $K_{p1} \leq K_{p2} \leq \dots \leq K_{pn}$
按此固有关系将上式记录序列重新排列为
 $\{ R_{p1}, R_{p2}, \dots, R_{pn} \}$

的操作称作**排序**。

排序

❖ 在待排记录序列中，任何两个关键字相同的记录，用某种排序方法排序后相对位置不变，则称这种排序方法是稳定的，否则称为不稳定的。

❖ 例如：

❖ 待排序列： **49**,38,65,97,76,13,27,49

❖ 排序后： 13,27,38,**49**,49,65,76,97 — 稳定

❖ 排序后： 13,27,38,49,**49**,65,76,97—不稳定

快速排序(Quick Sort)

- 快速排序算法是于1962年由英国的Tony Hoare 提出的，并于1980年获得Turing Awards
- 最坏运行时间： $O(n^2)$
- 平均运行时间： $O(n \log n)$
 - 实际运行效果最好的排序算法
 - 在 $O(n \log n)$ 中的常数因子非常小

快速排序

❖ 基本思想:

- 通过一趟排序将待排序记录分割成两个部分
- 一部分记录的关键字比另一部分的小。
- 选择一个关键字作为分割标准，称为**pivot**

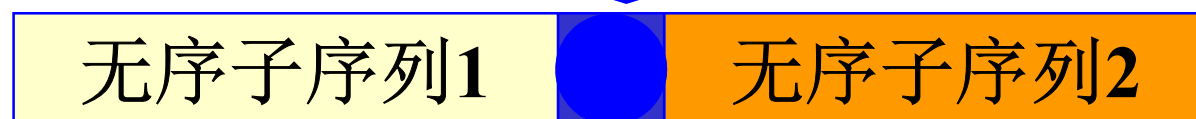
❖ 基本操作:

- 选定一记录**R** (**pivot**)，将所有其他记录关键字**k'**与该记录关键字**k**比较
 - 若 $k' < k$ 则将记录换至**R**之前;
 - 若 $k' > k$ 则将记录换至**R**之后;
- 继续对**R**前后两部分记录进行快速排序，直至排序范围为1;

快速排序

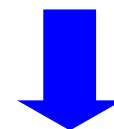
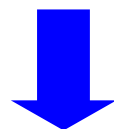


一次划分



49 38 65 97 76 13 27 49

27 38 13 **49** 76 97 65 49



继续划分

用第一个记录作pivot

49

49 38 65 97 76 13 27 49
l ↑ ↑ *h* ↑ ← *h*

27 38 65 97 76 13 49
l ↑ → *l* ↑ *h* ↑

27 38 97 76 13 65 49
l ↑ ↑ *h* ↑ ← *h* ↑

27 38 13 97 76 65 49
l ↑ → *l* ↑ *h* ↑

27 38 13 76 97 65 49
l ↑ *h* ↑ ← *h* ↑

27 38 13 49 76 97 65 49
l ↑ *h* ↑

一趟快速排序

快速排序---算法

轴(pivot) = 49

0	1	2	3	4	5	6	7
low 49	38	65	97	76	13	27	high 49
27	38	65	97	76	13	high 49	49
27	38	low 49	97	76	13	65	49
27	38	13	97	76	high 49	65	49
27	38	13	low 49	76	97	65	49
			high =low				

快速排序

[27 38 13] 49 [76 97 65] 49

一趟快速排序

[13] 27 [38] 49 [49 65] 76 [97]

两趟快速排序

13 27 38 49 49 65 76 97

三趟快速排序

快速排序

快速排序

❖ 设初始时

- **low**指针指向第一个记录;
- **high**指针指向最后一个记录;

❖ 一趟快速排序的算法过程:

1. 将第一个记录设置为**pivot**
2. 从表的两端交替地向中间扫描, 直到两个指针相遇
 - 先从高端扫描
 - 找到第一个比**pivotkey**小的记录
 - 将该记录移动到**low**指针指向的地方;
 - 再从低端扫描
3. 将**pivot**移动到**low**指针位置, 并返回该位置

快速排序

在快速排序中，记录的比较和交换是从两端向中间进行的，关键字较大的记录一次就能交换到后面单元，关键字较小的记录一次就能交换到前面单元，记录每次移动的距离较大，因而总的比较和移动次数较少。

快速排序—划分过程

快排序是一个分治算法

快排序的 **关键过程** 是每次递归的划分过程

划分问题描述：对一个序列，取它的一个元素作为轴，把所有小于轴的元素放在它的左边，大于它的元素放在它的右边

```
int Partition(SqList &L, int low, int high)
{ /*对顺序表L中子表r[low..high]的记录
  作一趟快速排序，并返回pivot记录所在位置。*/
```

```
  L.r[0]=L.r[low]; //用第一个记录作pivot记录
  pivotkey=L.r[low].key; // pivotkey是pivot关键字
```

```
  while(low<high)
  { //从表的两端交替地向中间扫描
    while(low<high && L.r[high]. Key>=pivotkey)
      --high;
    L.r[low]=L.r[high];
    while (low<high && L.r[low]. Key<=pivotkey)
      ++low;
    L.r[high]=L.r[low];      } // 交替扫描结束
```

```
  L.r[low]=L.r[0]; //pivot位置
  return low;      //返回pivot位置
```

```
}//Partition
```



```
void Qsort(SqList &L, int low, int high)
```

```
{//对顺序表L中的子序列L.r[low.. high]作快速排序
```

```
    if (low<high)
```

```
    {    pivotloc=Partition(L, low, high);
```

```
        QSort(L, low, pivotloc-1);
```

```
        Qsort(L, pivotloc+1, high);
```

```
    }
```

```
}
```

递归结束
条件

```
void QuickSort(SqList &L )
```

```
{//对顺序表L快速排序
```

```
    QSort(L, 1, L.length);
```

```
}
```

快速排序

- ❖ 快速排序的基本思想是基于分治策略的。对于输入的子序列 $L[p..r]$ ，分三步处理：
- ❖ **1、分解(Divide)**：将待排序列 $L[p..r]$ 划分为两个非空子序列 $L[p..q]$ 和 $L[q+1..r]$ ，使前面任一元素的值不大于后面元素的值。
- ❖ 途径实现：在序列 $L[p..r]$ 中选择数据元素 $L[q]$ ，经比较和移动后， $L[q]$ 将处于 $L[p..r]$ 中间的适当位置，使得数据元素 $L[q]$ 的值小于 $L[q+1..r]$ 中任一元素的值。

快速排序

- ❖ **2、递归求解(Conquer)**: 通过递归调用快速排序算法，分别对 $L[p..q]$ 和 $L[q+1..r]$ 进行排序。
- ❖ **3、合并(Merge)**: 由于对分解出的两个子序列的排序是就地进行的，所以在 $L[p..q]$ 和 $L[q+1..r]$ 都排好序后不需要执行任何计算 $L[p..r]$ 就已排好序，即自然合并。
- ❖ 这个解决流程是符合分治法的基本步骤的。因此，快速排序法是分治法的经典应用实例之一。

问题

- 请问快速排序的最好情形和最坏情形下的时间复杂度为多少？

时间复杂度分析

最坏情况分析：每次分成大小为1和n-1的两段

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ T(n-1) + O(n) & n > 1 \end{cases}$$

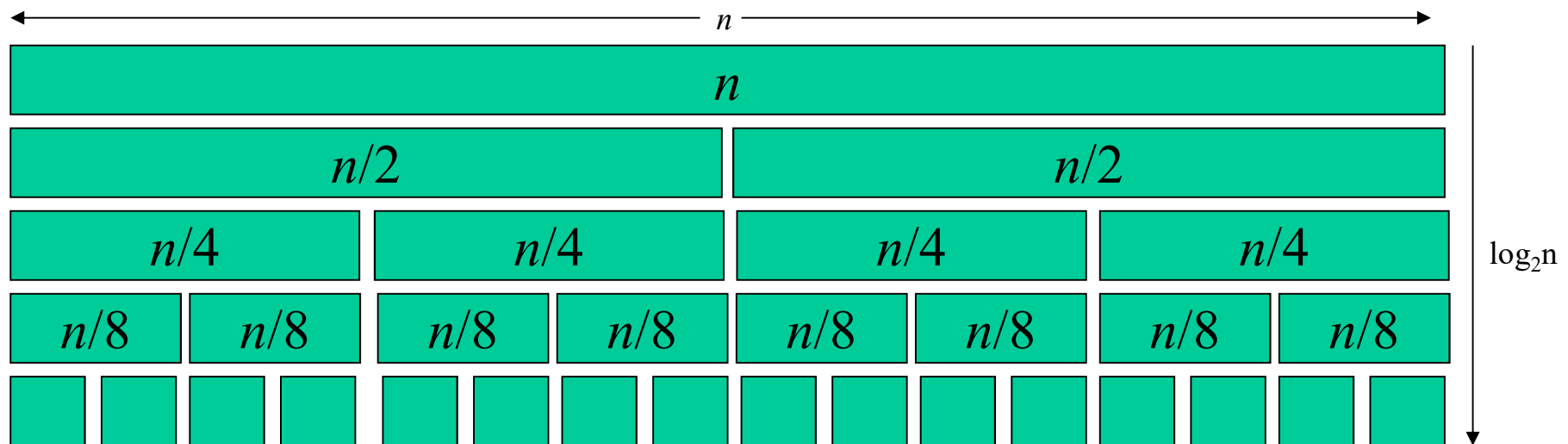
$$T(n) = O(n^2)$$

最好情况分析：每次分成大小为n/2的两段

$$T(n) = \begin{cases} O(1) & n \leq 2 \\ 2T(n/2) + O(n) & n > 2 \end{cases}$$

$$T(n) = O(n \log n)$$

算法分析—最好情形



Best Case Complexity $\rightarrow n \log n$

算法分析

- 注意：
用递归树来分析最好情况下的比较次数更简单。
- 因为每次划分后左、右子区间长度大致相等，故递归树的高度为 $O(\log n)$
- 递归树每一层上各结点所对应的划分过程中所需要的关键字比较次数总和不超过 n ，故整个排序过程所需要的关键字比较总次数 $C(n)=O(n\log n)$ 。

算法分析—最好情形

最好情况下，每次划分所取得基数都恰好为中值，即每次划分都产生两个大小约为 $n/2$ 的区域，因此**Partition**的计算时间 **$T(n)$** 满足

$$\begin{aligned}T(n) &= 2T(n/2) + \theta(n) \\ T(1) &= 0\end{aligned}$$

$$T(n) = O(n \log n)$$

$$T(n) = aT(n/c) + bn$$

$$T(n) = \begin{cases} \Theta(n) & a < c \\ \Theta(n \log n) & a = c \\ \Theta(n^{\log_c a}) & a > c \end{cases}$$

算法分析—最坏情形

当需要排序的序列已经有序时，快速排序需要做 **$n-1$** 次划分，每次划分需要的比较次数依次为 **$n-1, n-2, \dots, 1$** 。所以在这种情况下，快速排序的时间复杂度为

$$\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} \in O(n^2)$$

算法分析—最坏情形

- 快速排序算法的性能取决于划分的对称性。
- 最坏情况是每次划分选取的基数都是当前无序区中关键字最小(或最大)的记录

算法分析—最坏情形

- 最坏情况下划分产生的两个区域分别包含 **n-1** 个元素和 **1** 个元素。函数 **Partition** 的计算时间为 **O(n)**，如果 **Partition** 的每一步都出现这种不对称划分，则计算时间复杂性 **T(n)** 满足：

$$T(n) = \begin{cases} O(1) & n = 1 \\ T(n-1) + O(n) & n > 1 \end{cases}$$

解此递归方程可得 $T(n) = n^2$ 。

快速排序特点

❖ 存储结构：顺序

❖ 时间复杂度

– 最坏情况：每次划分选择是最小或最大元素

– 最坏情况： $O(n^2)$

– 最好情况（每次划分折半）： $O(n\log_2 n)$

– 平均时间复杂度为 $O(n\log_2 n)$

❖ 空间复杂度

– 最坏情况： $O(n)$

– 最好情况（每次划分折半）： $O(\log_2 n)$

– 平均空间复杂度 $O(\log_2 n)$

❖ 稳定性

– 不稳定

问题

- 选择不同的基数对排序的结果是否有影响？
- 基数的选择是否可以随机的？
- 选择不同的基数对排序的时间复杂度是否有影响？

算法优化—改进1

- 基数的选择

- 基本快速排序算法选择序列的第一个元素为基数，当基数使每次划分后的两个子序列长度接近时，算法效率很高，如果偏向某一边，则效率偏低

为了打破极端偏向某一边，可以用其他策略选择基数

- 选择 $\text{Array}[0]$, $\text{Array}[n-1]$, $\text{Array}[(n-1)/2]$ 三个数的中间值作为基数

算法优化—改进2

短序列排序：

- 快速排序对序列长度较大时很有效，但当序列较短时则效率不高，因为快排序是个递归算法，需要大量子过程调用
- 可以做如下改进：**当序列长度较大时采用快速排序算法。当序列长度被划分到小到一定长度时，采用较直接的算法（如插入排序）

快速排序

- 算法评论：对很长的序列来说，快速排序的平均性能最好。因此在很多软件，比如数据库系统中经常使用

算法优化—改进3

- 快速排序算法的性能取决于划分的对称性。
- 设计出采用随机选择策略的快速排序算法。
- 在快速排序算法的每一步中，当数组还没有被划分时，可以在 $a[p:r]$ 中随机选出一个元素作为划分基准，这样可以使划分基准的选择是随机的，从而可以期望划分是较对称的。

随机快速排序

- 取位于**low**和**high**之间的随机数 $k(\text{low} \leq k \leq \text{high})$ ，用 $R[k]$ 作为基准
- 选取基准最好的方法是用一个随机函数产生一个取位于**low**和**high**之间的随机数 $k(\text{low} \leq k \leq \text{high})$ ，用 $R[k]$ 作为基准，这相当于强迫 $R[\text{low}..\text{high}]$ 中的记录是随机分布的。用此方法所得到的快速排序一般称为**随机的快速排序**。

随机快速排序

- 注意：
- 随机化的快速排序与一般的快速排序算法差别很小。但随机化后，算法的性能大大地提高了，尤其是对初始有序的文件，一般不可能导致最坏情况的发生。
算法的随机化不仅仅适用于快速排序，也适用于其它需要数据随机分布的算法。

■ 算法分析

随机化的划分算法

```
int RandomizedPartition (Type a[], int p, int r)
{
    int i = Random(p,r);
    Swap(a[i], a[p]);
    return Partition (a, p, r);
}
```

📖 最坏时间复杂度: $O(n^2)$

📖 平均时间复杂度: $O(n \log n)$

📖 辅助空间: $O(n)$ 或 $O(\log n)$

归并排序

❖ 归并：

- 将两个或两个以上有序表组合成一个新的有序表。

❖ 2-路归并排序：

- 设初始序列含有 n 个记录，则可看成 n 个有序的子序列，每个子序列长度为1。
- 两两合并，得到 $\left\lfloor \frac{n}{2} \right\rfloor$ 个长度为 2 或 1 的有序子序列。
- 再两两合并，……如此重复，直至得到一个长度为 n 的有序序列为止。

归并排序

例

初始关键字: [49] [38] [65] [97] [76] [13] [27]



一趟归并后: [38 49] [65 97] [13 76] [27]



二趟归并后: [38 49 65 97] [13 27 76]



三趟归并后: [13 27 38 49 65 76 97]



归并排序

A: 1 3 8 9 11

B: 2 5 7 10 13

C: 1 2 3 5 7 8 9 10 11 13

合并排序

- 如何使用分治算法求解合并排序问题？

合并排序

- 合并算法排序即属于分治方法。合并（Merge）就是将两个或多个有序表合并成一个有序表。

基本思想： 将待排序元素分成大小大致相同的**2**个子集合，分别对**2**个子集合进行排序，最终将排好序的子集合合并成为所要求的排好序的集合。

归并排序 (Merge Sort)

归并排序是一个分治递归算法

- 递归基础：若序列只有一个元素，则它是有序的，不执行任何操作
- 递归步骤：
 - 先把序列划分成长度基本相等的两个序列
 - 对每个子序列递归排序
 - 把排好序的子序列归并成最后的结果

归并排序 (Merge Sort)

初始序列: [8, 4, 5, 6, 2, 1, 7, 3]

分解: [8, 4, 5, 6] [2, 1, 7, 3]

分解: [8, 4] [5, 6] [2, 1] [7, 3]

分解: [8] [4] [5] [6] [2] [1] [7] [3]

归并: [4, 8] [5, 6] [1, 2] [3, 7]

归并: [4, 5, 6, 8] [1, 2, 3, 7]

归并: [1, 2, 3, 4, 5, 6, 7, 8]

归并排序 (Merge Sort)

```
void MergeSort(Type a[], int left, int right)
{
    if (left < right)           //至少有2个元素
    {
        int i = (left + right) / 2; //取中点
        mergeSort(a, left, i);
        mergeSort(a, i + 1, right);
        merge(a, b, left, i, right); //合并到数组b
        copy(a, b, left, right);    //复制回数组a
    }
}
```

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + cn & n>2 \end{cases}$$

归并排序

```
void Merge(T A[], int Alen, T B[], int Blen, T C[]){  
    int i=0,j=0,k=0;  
    while(i < Alen && j < Blen){  
        if(A[i] < B[j])  
            C[k++] = A[i++];  
        else  
            C[k++] = B[j++];  
    }  
    while(i < Alen)  
        C[k++] = A[i++];  
    while(j < Blen)  
        C[k++] = B[j++];  
}
```

归并排序

```
void MergeSort(int A[], int B[], int l, int h)
{
    if(l == h)
        return;
    int m = (l+h)/2;
    MergeSort(A, B, l, m);
    MergeSort(A, B, m+1, h);
    Merge(A, B, l, m, h);
}
```

$$T(n) = \begin{cases} 1 & n=2 \\ 2T(n/2) + cn & n>2 \end{cases}$$

归并排序

- ❖ 时间复杂度:

- 共进行 $\lceil \log_2 n \rceil$ 趟归并, 每趟对 n 个记录进行归并
- 所以时间复杂度是 $O(n \log n)$

- ❖ 空间复杂度:

- $O(n)$

- ❖ 稳定性:

- 稳定

$$T(n) = aT(n/c) + bn \quad (\text{即 } x=1)$$

$$T(n) = \begin{cases} \Theta(n) & a < c \\ \Theta(n \log n) & a = c \\ \Theta(n^{\log_c a}) & a > c \end{cases}$$

- 一个问题平分为两个子问题

$$\Theta(n \log n)$$

求下列递归方程的值

- $T(n)=3T(n/2)+bn$

$$T(n) = \Theta(n^{\log_2 3})$$

- $T(n)=4T(n/2)+bn$

$$T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

- $T(n)=8T(n/2)+bn$

$$T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

- 子问题的大小还是 **$n/2$** ，而子问题的个数是分别是 **3、4、8**，则这些问题的时间复杂度分别为：

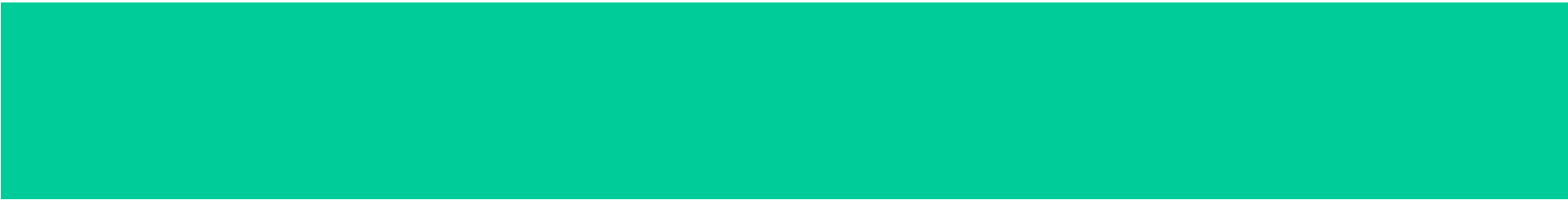
$$\Theta(n^{\log_2 3}) \quad \Theta(n^2) \quad \Theta(n^3)$$

求下列递归方程的值

- $T(n)=8T(n/2)+bn$ $T(n) = \Theta(n^{\log_2 8})$
- $T(n)=8T(n/4)+bn$ $T(n) = \Theta(n^{\log_4 8})$
- $T(n)=8T(n/8)+bn$ $T(n) = \Theta(n^{\log_8 8}) = \Theta(n)$

■ 子问题的规模不同，子问题的个数相同都是**8**，则这些问题的时间复杂度分别为：

$$\Theta(n^3) \quad \Theta(n^{\log_4 8}) \quad \Theta(n)$$



应用3：循环赛日程表

■应用：循环赛日程表

分治法不仅可以用来设计算法, 而且在其他方面也有广泛应用。例如可以用分治思想来设计电路、构造数学证明等。现举一例加以说明。

循环赛日程表

设计一个满足以下要求的比赛日程表（ 2^k 个选手）：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) 循环赛一共进行 $n-1$ 天。

循环赛日程表

下图所列出的正方形表是4个选手的比赛日程表。其中左上角与左下角的两小块分别为选手1至选手2和选手3至选手4第1天的比赛日程。据此，将左上角小块中的所有数字按其相对位置抄到右下角，将左下角小块中的所有数字按其相对位置抄到右上角，这样我们就分别安排好了选手1至选手2和选手3至选手4在后2天的比赛日程。这种安排是符合要求的。

1	2	3	4
2	1	4	3
3	4	1	2
4	3	2	1

4个选手的比赛日程表

问题

- 如何对问题进行分解？

循环赛日程表

按分治策略，将所有的选手分为两半， n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。

递归地用对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

循环赛日程表

1	2
2	1

(a) $2^k(k=1)$ 个选手比赛



1 2	3 4
2 1	4 3
3 4	1 2
4 3	2 1

(b) $2^k(k=2)$ 个选手比赛



1 2 3 4	5 6 7 8
2 1 4 3	6 5 8 7
3 4 1 2	7 8 5 6
4 3 2 1	8 7 6 5
5 6 7 8	1 2 3 4
6 5 8 7	2 1 4 3
7 8 5 6	3 4 1 2
8 7 6 5	4 3 2 1

(c) $2^k(k=3)$ 个选手比赛

循环赛日程表

按分治策略，将所有的选手分为两半， n 个选手的比赛日程表就可以通过为 $n/2$ 个选手设计的比赛日程表来决定。递归地用对选手进行分割，直到只剩下2个选手时，比赛日程表的制定就变得很简单。这时只要让这2个选手进行比赛就可以了。

1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

循环赛日程表的推广

设计一个满足以下要求的比赛日程表：

- (1) 每个选手必须与其他 $n-1$ 个选手各赛一次；
- (2) 每个选手一天只能赛一次；
- (3) n 为偶数时，循环赛一共进行 $n-1$ 天。
 n 为奇数时，循环赛一共进行 n 天。

循环赛日程表

	第1天	第2天	第3天
1			
2			
3			
4			

循环赛日程表

	第1天	第2天	第3天
1			
2			
3			

	1	2	3
第1天	2	1	-
第2天	3	-	1
第3天	-	3	2

	第1天	第2天	第3天
1	2	3	-
2	1	-	3
3	-	1	2

循环赛日程表

	第1天	第2天	第3天	第4天	第5天
1					
2					
3					
4					
5					
6					

循环赛日程表

	1	2	3	4	5
第1天	2	1	-	5	4
第2天	3	5	1	-	2
第3天	4	3	2	1	-
第4天	5	-	4	3	1
第5天	-	4	5	2	3

	1	2	3	4	5	6	7	8	9	10
第1天	2	1	8	5	4	7	6	3	10	9
第2天	3	5	1	9	2	8	10	6	4	7
第3天	4	3	2	1	10	9	8	7	6	5
第4天	5	7	4	3	1	10	2	9	8	6
第5天	6	4	5	2	3	1	9	10	7	8
第6天	7	8	9	10	6	5	1	2	3	4
第7天	8	9	10	6	7	4	5	1	2	3
第8天	9	10	6	7	8	3	4	5	1	2
第9天	10	6	7	8	9	2	3	4	5	1